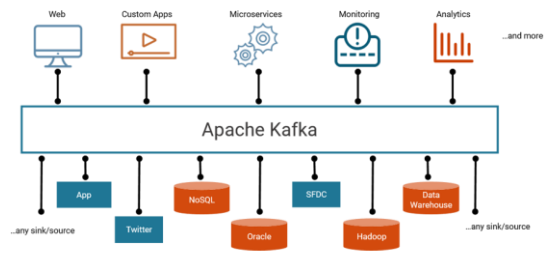
Big Data
(MEI)

João Pedro Costa
jcosta@isec.pt

1

Kafka

- an open-source distributed event streaming platform
- publish-subscribe messaging rethought as a distributed, partitioned, replicated commit log service
- used by thousands of companies for high-performance data pipelines, streaming analytics, data integration, and mission-critical applications.



2

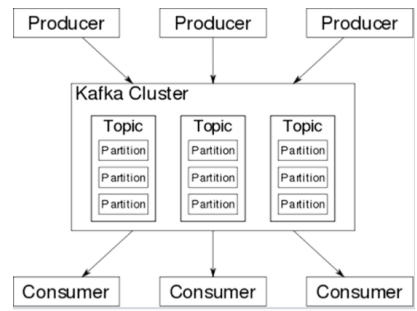
Kafka

- **High Throughput**
  - Deliver messages at network limited throughput using a cluster of machines with latencies as low as 2ms
- **Scalable**
  - Scale clusters up to a thousand brokers, trillions of messages per day, petabytes of data, hundreds of thousands of partitions
  - Elastically expand and contract storage and processing
- **Permanent storage**
  - Store streams of data safely in a distributed, durable, fault-tolerant cluster
- **High availability**
  - Stretch clusters efficiently over availability zones or connect separate clusters across geographic regions.

3

Kafka - concepts

- **Producers**
  - client applications that publish (write) events to Kafka
- **Consumers**
  - are those that subscribe to (read and process) these events.



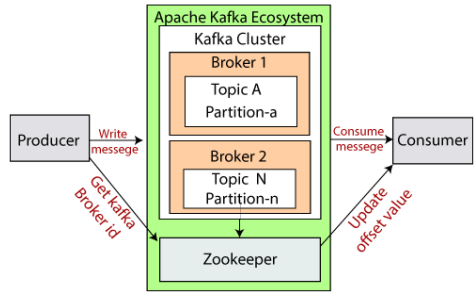
- producers and consumers are fully decoupled and agnostic of each other
- a key design element to achieve the high scalability
  - e.g. producers never need to wait for consumers

4

Kafka

combines three key capabilities for event streaming end-to-end

- **publish** (write) and subscribe to (read) streams of events, including continuous import/export of your data from other systems
- **store** streams of events durably and reliably for as long as you want
- **process** streams of events as they occur or retrospectively



5

Kafka - concepts

- **event**
  - records the fact that "something happened"
  - also called record or message
  - when you read or write data to Kafka, you do this in the form of events
  - has a key, value, timestamp, and optional metadata header
    - *Event key: "Alice"*
    - *Event value: "Made a payment of $200 to Bob"*
    - *Event timestamp: "Jun. 25, 2020 at 2:06 p.m."*
- events are organized and durably stored in **topics**

6

Kafka - concepts

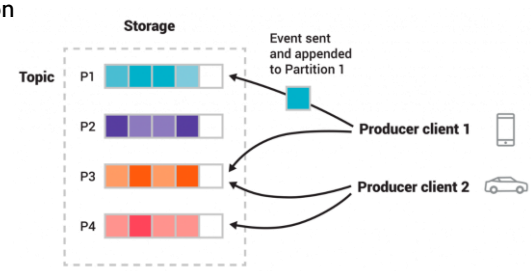
Topics

- very simplified, a topic is similar to a folder in a filesystem, and the events are the files in that folder
- topics are **partitioned**, meaning a topic is spread over a number of "buckets" located on different Kafka brokers
- this distributed placement of your data is very important for scalability because it allows client applications to both read and write the data from/to many brokers at the same time
- when a new event is published to a topic, it is actually appended to one of the topic's partitions
- topic is divided into a set of totally ordered partitions

Kafka - concepts

Topic Partitions

- when a new event is published to a topic, it is appended to one of the topic's partitions
- events with the same event key (e.g., a customer or vehicle ID) are written to the same partition

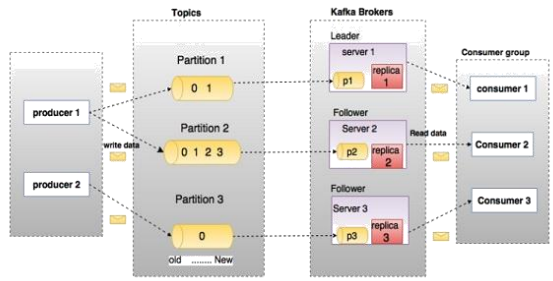


Events with the same key (same color) are written to the same partition.
Note that both producers can write to the same partition if appropriate.

Kafka - concepts

Topic Partitions

- Kafka guarantees that any consumer of a given topic-partition will always read that partition's events in exactly the same order as they were written
- every topic can be replicated, for data fault-tolerant and highly-available, so that there are always multiple brokers that have a copy of the data
- the replication is performed at the level of topic-partitions



Kafka - concepts

Producers

- push data into the leader broker
  - each partition in Kafka has a single leader and zero or more followers
  - the number of replicas including the leader constitute the replication factor
- all writes go to the leader of the partition
- typically, there are many more partitions than brokers and the leaders are evenly distributed among brokers
- followers' logs are identical to the leader's log
  - all have the same offsets and messages in the same order

Kafka - concepts



Consumers

- pull data from the broker
- reads only once the same data from the broker
- reads can go to the leader or the followers of the partition
- each consumer, logs its position of what it has consumed in each partition, or the offset of the next message
- with a simple integer stores the consumer state, what has been consumed, and can be periodically checkpointed, and easily acknowledge

Kafka



- Install

```
$ curl -fSL https://downloads.apache.org/kafka/3.7.0/kafka_2.13-3.4.0.tgz |
tar -xz -C kafka
$ cd kafka
```

- Startup and running

```
# start Zookeeper
nohup bin/zookeeper-server-start.sh config/zookeeper.properties &

# Start the Kafka broker service
bin/kafka-server-start.sh config/server.properties
```

Kafka

- Get kafka docker image

```
docker pull apache/kafka:3.7.0
```

- Run the docker container

```
docker run -p 9092:9092 apache/kafka:3.7.0
```


14

Kafka

- To create a topic

```
bin/kafka-topics.sh --create --topic sample --partitions 10
--replication-factor 1 --bootstrap-server 10.204.131.11:9092
```

- To list topics

```
bin/kafka-topics.sh --list --bootstrap-server 10.204.131.11:9092
```


16

Kafka

- To add partitions (10 → 5)

```
bin/kafka-topics.sh --alter --topic sample --partitions 5
--bootstrap-server 10.204.131.11:9092
```

- To delete a Topic

```
bin/kafka-topics.sh --delete  --topic sample
--bootstrap-server 10.204.131.11:9092
```


17

Kafka

- Write events into the topic

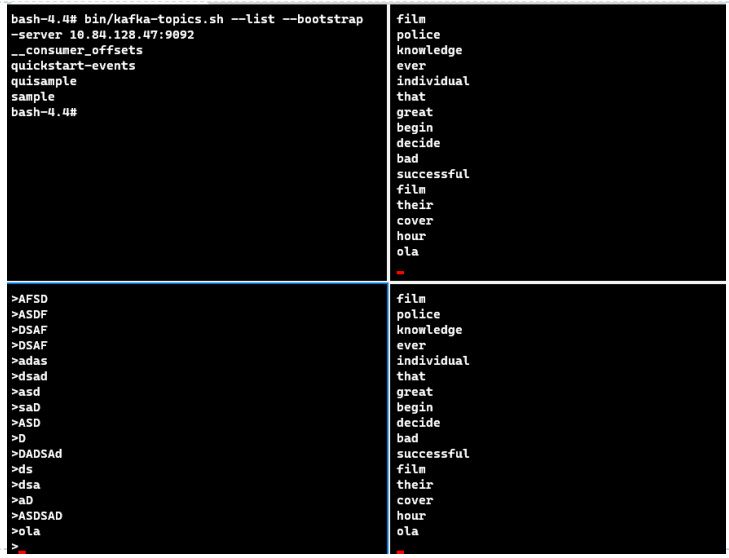
```
bin/kafka-console-producer.sh --topic sample
--bootstrap-server 10.204.131.11:9092
```

- Read the events

```
bin/kafka-console-consumer.sh --topic sample
--from-beginning --bootstrap-server 10.204.131.11:9092
```


18

Kafka



Python packages to Kafka

- Access Kafka from Python

```
pip install kafka-python
```

- Other alternative libraries

```
pip install pykafka

pip install confluent-kafka
```

Kafka - Producer

- Create a producer and send a message

```
# Producer to a given topic
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='10.204.131.11:9092')

producer.send('sample', b'Hello, World!')
```

```
# Producer to a given topic
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='10.204.131.11:9092')

producer.send('sample', key=b'message-two', value=b'This is Kafka-Python')
```


Kafka - Producer

- Create a producer and send a message

```
# Producer to a given topic
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='10.84.128.47:9092')

producer.send('sample', b'Hello, World!')
```

```
<kafka.producer.future.FutureRecordMetadata at 0x7f5ea4291490>
```

```
producer.send('sample', key=b'message-two', value=b'This is Kafka-Python')
```

```
<kafka.producer.future.FutureRecordMetadata at 0x7f5ea42907d0>
```

```
for x in range(1,100):
    producer.send('sample', bytes('python ' + str(x),"utf-8"))
print("produced 100 rows")
```

```
produced 100 rows
```

Kafka

- Kafka Consumer

```
#Consume a topic
from kafka import KafkaConsumer
consumer = KafkaConsumer('sample')

for message in consumer:
    print (message)
```

```
#Consume a topic
from kafka import KafkaConsumer
consumer = KafkaConsumer('sample',bootstrap_servers='10.204.131.11:9092')

for message in consumer:
    print (message)
```


Kafka

- Kafka Consumer

```
#Consume a topic
from kafka import KafkaConsumer
consumer = KafkaConsumer('sample',bootstrap_servers='10.84.128.47:9092')
consumer
```

```
<kafka.consumer.group.KafkaConsumer at 0x7f1a2fe2b850>
```

```
for message in consumer:
    print (message)
    print(message.timestamp)
    print(message.key)
    print(message.value)
```

Kafka - Producer

- Create a producer and send json

```
# Serialize json messages
import json

# Producer to a given topic
from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers='10.204.131.11:9092',
                         value_serializer=lambda v: json.dumps(v) \
                    .encode('utf-8'))

producer.send('sample', {'foo': 'bar'})
```


Kafka - Producer

- Create a producer and send json

```
import json
# Producer to a given topic
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='10.84.128.47:9092',
        value_serializer=lambda v: json.dumps(v) .encode('utf-8'))
```

```
producer.send('sample', {'foo': 'bar'})
```

```
<kafka.producer.future.FutureRecordMetadata at 0x7fc734ab0c10>
```

```
producer.close()
```

```
cover
hour
ola
{"foo": "bar"}
```

Kafka - Producer

- Compress messages

```
# Producer to a given topic
from kafka import KafkaProducer

# Compress messages
producer = KafkaProducer( bootstrap_servers='10.204.131.11:9092',
                          compression_type='gzip')
for i in range(1000):
    producer.send('foobar', b'msg %d' % i)
```


27

Kafka - example

```
# Producer to a given topic
from kafka import KafkaProducer

# Compress messages
producer = KafkaProducer(bootstrap_servers='10.204.131.11:9092')

for i in range(1000):
    producer.send('foobar', b'msg %d' % i)
```

```
from kafka import KafkaConsumer
consumer = KafkaConsumer('foobar',bootstrap_servers='10.204.131.11:9092')

for message in consumer:
    print (message)
    print (message.value)
```


28

Kafka + Streaming integration

- Run spark submit spark-sql-kafka packages

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1
        pythonfile.py
```

- Example: to submit the python file `kafka1.py`

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 kafka1.py
```


30

Kafka - Consumer

- Subscribe to 1 topic

```
# Subscribe to 1 topic
df = spark \
  .readStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .option("subscribe", "topic1") \
  .load()
df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)")
```

```
# Subscribe to 1 topic, with headers
df = spark \
  .readStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .option("subscribe", "topic1") \
  .option("includeHeaders", "true") \
  .load()
df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)", "headers")
```


31

Kafka - Consumer

- Subscribe to multiple topics

```
# Subscribe to multiple topics
df = spark \
  .readStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .option("subscribe", "topic1,topic2") \
  .load()
df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)")
```

```
# Subscribe to a pattern
df = spark \
  .readStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .option("subscribePattern", "topic.*") \
  .load()
df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)")
```


Kafka

```
df = spark.readStream.format("kafka") \
  .option("kafka.bootstrap.servers", "10.204.131.11:9092") \
  .option("subscribe", "sample") \
  .load()
```

Each row in the source has the following schema:

Column	Type
key	binary
value	binary
topic	string
partition	int
offset	long
timestamp	timestamp
timestampType	int
headers (optional)	array

Kafka - Producer

- Creating a Kafka Sink for Streaming Queries

```
# Write key-value data from a DataFrame to a Kafka topic specified in an option
ds = df \
  .selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)") \
  .writeStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .option("topic", "topic1") \
  .start()
```

```
#Write key-value data from a DataFrame to Kafka using a topic specified in the data
ds = df \
  .selectExpr("topic", "CAST(key AS STRING)", "CAST(value AS STRING)") \
  .writeStream \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .start()
```


Kafka - Producer

- Writing the output of Batch Queries to Kafka

```
# Write key-value data from a DataFrame to a Kafka topic specified in an option
df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)") \
  .write \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .option("topic", "topic1") \
  .save()
```

```
# Write key-value data from a DataFrame to Kafka to a topic specified in the data
df.selectExpr("topic", "CAST(key AS STRING)", "CAST(value AS STRING)") \
  .write \
  .format("kafka") \
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2") \
  .save()
```

Kafka

```
df = spark.readStream.format("kafka") \
  .option("kafka.bootstrap.servers", "10.204.131.11:9092") \
  .option("subscribe", "sample") \
  .load()
```